Практичне завдання 4

Підготовка програмного проєкту до збереження у GitHub

Дисципліна: Інформатика. Завдання з ІТ.
Тема програмного продукту: Система обліку навчальних завдань студента.
Дата підготовки: 9 листопада 2026 року.

Мета та отриманий результат

Мета роботи — організувати програмні засоби навчального проєкту у локальному репозиторії Git, підготувати супровідну документацію, виключити локальні дані з історії та визначити порядок публікації на GitHub.

Створено застосунок для планування навчальних завдань. Підготовлено сирцевий код, файл .gitignore, README.md, конфігурацію залежностей, архітектурну схему та автоматизовані перевірки. Локальну історію репозиторію передано у файлі student-task-manager.bundle; основна гілка — main.

Стан виконання вимог

Вимога завдання	Результат
Працездатна реалізація і структура	Код і конфігурацію підготовлено; обробник запитів перевірено автоматично
README та .gitignore	Підготовлено, локальні дані виключено
Локальна гілка main і початковий коміт	Підготовлено та передано в Git bundle
Віддалений репозиторій і git push	Опубліковано вебінтерфейсом; git push не виконувався
Публічний доступ для комісії	Репозиторій має видимість Public

Публічний репозиторій: https://github.com/student12210/student-task-manager. Власник — student12210, основна гілка — main. Код і документацію завантажено через вебінтерфейс GitHub з фіксацією змін. Команда git push під час публікації не виконувалася; вебзавантаження не є підтвердженням її виконання.

1 Призначення та функції програми

Система призначена для індивідуального використання студентом. Для кожного завдання можна задати назву, предмет, опис, термін виконання та пріоритет. Завдання можна редагувати, видаляти після підтвердження, позначати виконаним і повертати до роботи.

Пошук працює за назвою, предметом та описом. Фільтри відображають усі, активні, виконані або прострочені завдання. Лічильники показують кількість записів у кожній категорії. Простроченим є невиконане завдання, дата якого передує поточному локальному дню; строк на сьогодні не вважається простроченим.

Технологічний стек

Компонент	Технологія та роль
Клієнтська частина	HTML, CSS, JavaScript; форма, список, пошук і фільтри
Серверна частина	Node.js 22 або новіший; стандартний модуль node:http
Збереження	JSON файл на локальному диску; стандартний node:fs
Тести	Стандартний модуль node:test та assert
Керування версіями	Git, гілка main; публічний репозиторій GitHub

Сторонні пакети не потрібні. package.json визначає версію продукту, мінімальну версію Node.js, команди запуску та тестування. package-lock.json містить узгоджений опис конфігурації npm. Окремий сервер бази даних або обліковий запис для локальної роботи не потрібні.

Архітектура

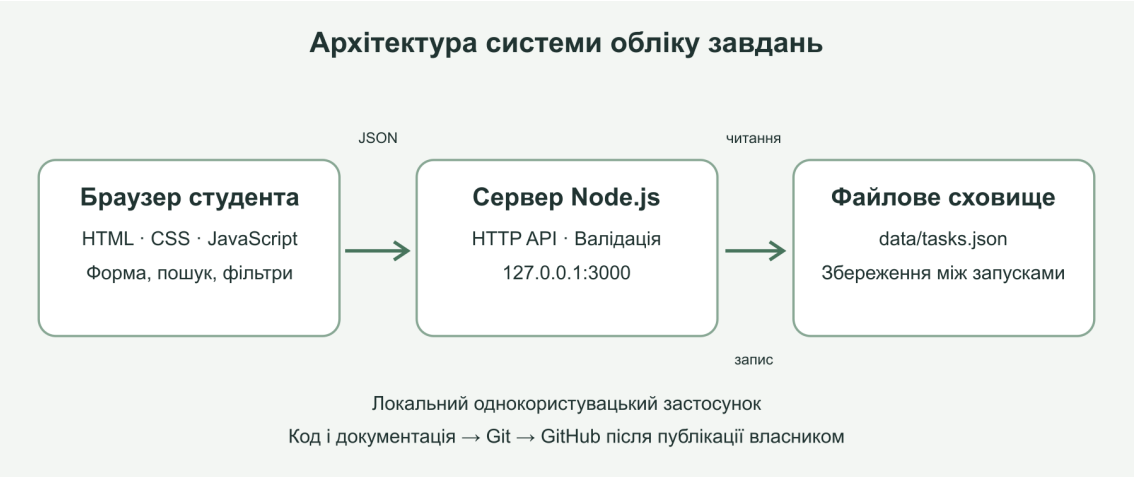


Рисунок 1 — Архітектура локальної системи обліку навчальних завдань

Браузер отримує сторінку від локального сервера та обмінюється з ним JSON даними. Сервер перевіряє введення і зберігає завдання на диску. Застосунок розрахований на одного користувача та один запущений процес сервера.

2 Структура та документація репозиторію

Шлях	Призначення
src/server.js	Обробка запитів, валідація, читання і збереження
public/index.html	HTML форма та список завдань
public/app.js	Клієнтська логіка і взаємодія з API
public/styles.css	Оформлення для комп’ютера і мобільного екрана
test/app.test.js	Перевірки обробника запитів та збереження
docs/architecture.svg і .png	Архітектурна схема у векторному та растровому форматах
docs/test-results.txt	Протокол виконання автоматизованих тестів
docs/PUBLISH.md	Порядок публікації на GitHub
README.md	Опис, стек, структура та інструкція запуску
package.json і package-lock.json	Команди та опис залежностей
.gitignore	Виключення локальних даних, кешу і секретів
start.cmd	Запуск програми у Windows

Виключення файлів з історії Git

У .gitignore зазначено папки data, node_modules, dist, coverage, особисті налаштування IDE, файли .env, журнали та тимчасові файли. Файл data/tasks.json утворюється під час роботи і не входить до репозиторію. Завдяки цьому початкове встановлення створює порожній список, а локальні завдання не опиняються у публічному доступі.

README містить опис функцій, інструкцію запуску, команди тестування, схему архітектури, опис API і відомості про збереження даних. Для резервного копіювання потрібно зупинити сервер і скопіювати data/tasks.json. У разі пошкодження JSON програма завершує запуск з помилкою, не замінюючи файл порожнім масивом.

Обмін даними

API підтримує GET /api/tasks, POST /api/tasks, PUT /api/tasks/:id і DELETE /api/tasks/:id. Сервер призначає унікальний ідентифікатор, дату створення і дату оновлення. Некоректні назви, дати, статуси й пріоритети відхиляються. Текст користувача відображається через textContent, тому не інтерпретується як HTML.

3 Запуск і перевірка програмного продукту

Необхідні інструменти: Node.js 22 або новіший, браузер і Git. У середовищі підготовки визначено Node.js 24.22.1 та Git 2.56.0.windows.4. Для запуску потрібно відкрити термінал у папці student-task-manager і виконати команду:

```
node src/server.js
```

Після повідомлення про запуск відкрийте http://127.0.0.1:3000. Альтернатива — npm start або start.cmd у Windows. Якщо npm недоступний, прямий запуск через node не потребує npm. Для зупинки застосовується Ctrl+C. Сервер прив’язаний до 127.0.0.1 і не призначений для відкриття у зовнішню мережу.

Результати автоматизованої перевірки

Команда node --test виконала 9 предметних перевірок і зовнішню тестову групу: загалом 10 успішних результатів, 0 помилок. Синтаксис src/server.js і public/app.js додатково перевірено командою node --check.

Перевірка	Результат
Порожній початковий список	Пройдено
Створення запису українською мовою	Пройдено
Редагування та зміна статусу	Пройдено
Читання збереженого після повторного створення застосунку	Пройдено
Порожні назви, неіснуючі дати та некоректні значення	Відхилено як очікувалося
Некоректний JSON	Відхилено як очікувалося
Запит зміни зі стороннього джерела	Відхилено як очікувалося
Видача сторінки, скрипту, стилів і невідомі маршрути	Пройдено
Видалення запису	Пройдено

Тести викликають реальний обробник запитів у пам’яті та працюють зі справжнім тимчасовим файлом. TCP з’єднання у середовищі перевірки блокуються помилкою EACCES, тому наскрізний сценарій браузер — локальний сервер тут не підтверджений. Успішний запуск слухача сервера зафіксовано, але він не замінює перевірку взаємодії через браузер.

Перед поданням доцільно локально пройти сценарій: додати завдання, змінити його назву, знайти через пошук, позначити виконаним, перезапустити програму і перевірити збереження; після цього видалити запис. Повний протокол автоматизованих тестів наведено у docs/test-results.txt.

4 Публікація на GitHub

Після входу власника у GitHub створено публічний репозиторій student12210/student-task-manager. Основна гілка — main. Адреса репозиторію: https://github.com/student12210/student-task-manager.

Через форму Upload files завантажено конфігурацію, README, серверний код, клієнтську частину, тести, схему та звіт. Структура папок збережена. Зміни зафіксовано комітами безпосередньо у main. Папку data, персональні налаштування IDE, паролі й токени до репозиторію не додавали.

Локальний репозиторій також має налаштований origin на опубліковану адресу. У комплекті залишено Git bundle із локальною історією підготовки. Локальна та вебісторія мають різне походження. Для подальшої роботи з публікацією доцільно клонувати репозиторій GitHub:

```
git clone https://github.com/student12210/student-task-manager.git
cd student-task-manager
node src/server.js
```

Вебінтерфейс підтверджує видимість Public та гілку main. Посилання доступне для передання екзаменаційній комісії. README пояснює запуск і тестування; схема відображає архітектуру програмного продукту.

Командний спосіб для наступних змін

Під час цієї публікації команда git push не виконувалася: файли передані вебформою GitHub. Якщо викладач вимагає саме цей крок у терміналі, після клонування й власного редагування потрібно виконати [1]:

```
git add .
git commit -m "Update student task manager"
git push origin main
```

Висновки

Створено програму обліку завдань, супровідні документи, архітектурну схему та локальну історію Git. Програмні перевірки основних операцій і збереження даних пройдено. Код із документацією опубліковано в main публічного репозиторію GitHub. Обмеження перевірки браузерної взаємодії та відсутність виконання git push зазначено окремо.

Використані джерела

1. GitHub Docs. Adding locally hosted code to GitHub. https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github

2. Git. Reference documentation. https://git-scm.com/docs

3. Node.js. HTTP. https://nodejs.org/api/http.html

4. Node.js. Test runner. https://nodejs.org/api/test.html